ĐẠI HỌC ĐÀ NẴNG
TRƯỜNG DẠI HỌC SƯ PHẠM
KHOA TOÁN – TIN

BÁO CÁO BÀI TẬP THỰC HÀNH NHÓM SỐ 2

Học phân: Lập trình mạng

ĐỀ TÀI:

Xây dựng cặp echo server – client TCP, cài đặt bước đăng nhập của giao thức chat và công cụ tra cứu tên miền

Nhóm thực hiện : 09
Thành viên : 1. Phimmasone Khamphouvanh : 3120224192
2. Bounsavath Ameena : 3120224190
3. Lattanaphimphone Chindaphet : 3120224191
4. Silaphet Thit : 3120224188
5. Thepsoulivong Syvone : 3120224193
Giảng viên hướng dẫn : Nguyễn Hoàng Hải

Đà Nẵng,tháng 9 năm 2026

MỤC LỤC

Phân - A. Lập trình echo client–server 3
A.1. Mục tiêu 3
A.2. Môi trường và cấu hình thử nghiệm 3
A.3. Kết quả chạy Echo Server và Echo Client 3
A.3.1. Khởi động Server 3
A.3.2. Client kết nối đến Server 3
A.4. Thông tin Socket 4
A.5. Kiểm tra kết nối bằng netstat 4
A.6. Thử nghiệm nhiều Client Session 5
A.7. Thử nghiệm lỗi khi Client chạy trước Server 5
A.8. Thử nghiệm bỏ SO_REUSEADDR 6
A.9. Thử nghiệm UTF-8 và dữ liệu khoảng 5000 ký tự 6
A.9.1. Chuỗi tiếng Việt 6
A.9.2. Dữ liệu khoảng 5000 ký tự 6
A.10. Tổng kết Part A 7
Phần B - Cài đặt bước đăng nhập của giao thức chat 7
B.1. Mục tiêu 7
B.2. Đặc tả thông điệp 8
B.3. Cài đặt Server 8
B.4. Cài đặt Client 9
B.5. Xử lý ranh giới thông điệp 9
B.6. Kiểm thử và nhận xét. 10
B.7. Kiểm thử hai client cùng kết nối 12
Phần C – Công cụ tra cứu tên miền dns_lookup.py 13
C.1 Mục tiêu 13
C.2 Nội dung thực hiện 13
C.3 Kết quả mã nguồn chương trình 14
C.4 Kết quả chạy chương trình 14
C.5 Kiểm tra tên miền không tồn tại 14
Phần D – Câu hỏi thảo luận 15
KẾT LUẬN 17
TÀI LIỆU THAM KHẢO 18
PHỤ LỤC 19

Quy cách trình bày: Font chữ: Times New Roman, Cỡ chữ:13**,** Căn lề**:** Top:3; Bottom: 2; Left: 3; Right: 2.

Phân - A. Lập trình echo client–server

A.1. Mục tiêu

Mục tiêu của phần thực hành này là xây dựng chương trình Echo Client–Server sử dụng TCP Socket trong Python [1]. Client gửi dữ liệu đến Server, Server nhận và gửi lại chính dữ liệu đó cho Client.

Chương trình sử dụng cổng 9009 theo quy ước của nhóm 09. Server sử dụng socket TCP để lắng nghe kết nối và xử lý từng phiên Client.

A.2. Môi trường và cấu hình thử nghiệm

Thành phần	Thông tih
Nhóm	Nhóm09
Port	9009
Server IP	172.20.10.3
Client IP	172.20.10.9
Giao thức	TCP / IPv4
Python	3.14.7
Hệ điều hành	Windows
Buffer Server	1024 bytes
Timeout Client	5 giây

Vai trò

- **Server:** máy có địa chỉ IP 172.20.10.3
- **Client:** máy có địa chỉ IP 172.20.10.9

Server sử dụng 0.0.0.0:9009 để lắng nghe kết nối trên các interface IPv4.

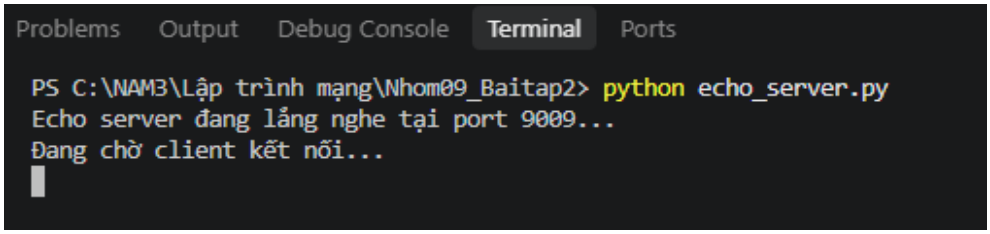
A.3. Kết quả chạy Echo Server và Echo Client

A.3.1. Khởi động Server

Server được chạy bằng lệnh:

- `python echo_server.py`

Server hiển thị:



Hình A.3.1. Server khởi động và lắng nghe tại port 9009.

A.3.2. Client kết nối đến Server

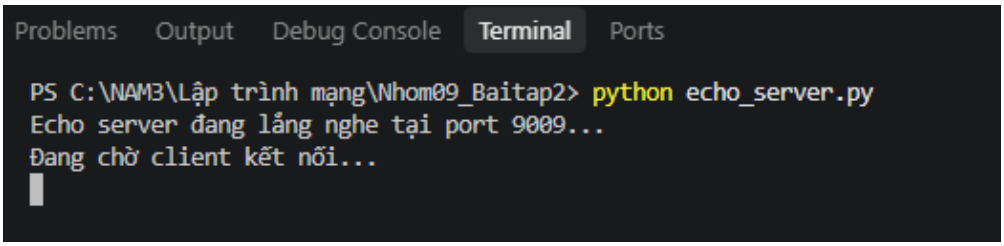
Client được chạy bằng:

- `python echo_client.py`

Client kết nối thành công đến:

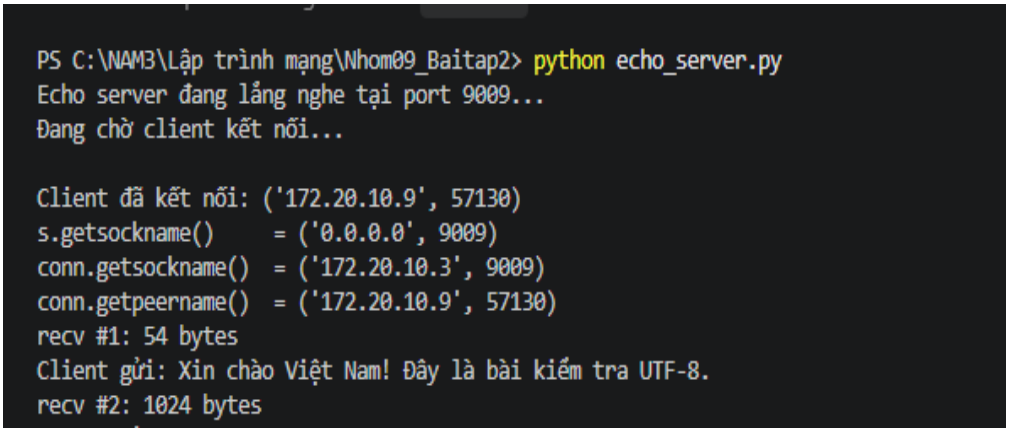
- 172.20.10.3:9009

Kết quả hiển thị:



Hình A.3.2. Client kết nối thành công đến Echo Server.

A.4. Thông tin Socket



Hình A.4. Thông tin địa chỉ của listening socket và session socket.

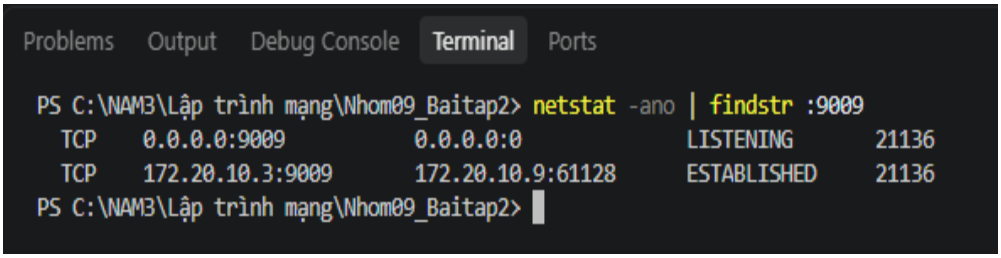
Trong đó:

`s.getsockname()` cho biết địa chỉ của listening socket, trong khi `conn.getsockname()` cho biết địa chỉ local của session socket và `conn.getpeername()` cho biết địa chỉ của phía Client [1]. Client sử dụng port tạm thời 57130, trong khi Server sử dụng port 9009.

A.5. Kiểm tra kết nối bằng netstat

Sử dụng lệnh: `netstat -ano | findstr :9009`

Kết quả:



Hình A.5. Kết quả kiểm tra kết nối bằng netstat.

Kết quả cho thấy Server đang ở trạng thái LISTENING trên port 9009. Đồng thời có một kết nối TCP ở trạng thái ESTABLISHED giữa Server 172.20.10.3:9009 và Client 172.20.10.9:61128.

Bảng 5-tuple

Thành phần	Giá trị
Protocol	TCP
Source IP	172.20.10.9
Source Port	61128
Destination IP	172.20.10.3
Destination Port	9009
State	ESTABLISHED

A.6. Thử nghiệm nhiều Client Session

Server đã xử lý hai phiên kết nối TCP liên tiếp.

Phiên thứ nhất:

```
PS C:\NAM3\Lập trình mạng\Nhom09_Baitap2> python echo_server.py
Echo server đang lắng nghe tại port 9009...
Đang chờ client kết nối...

Client đã kết nối: ('172.20.10.9', 49248)
s.getsockname()     = ('0.0.0.0', 9009)
conn.getsockname()  = ('172.20.10.3', 9009)
conn.getpeername()  = ('172.20.10.9', 49248)
recv #1: 54 bytes
Client gửi: Xin chào Việt Nam! Đây là bài kiểm tra UTF-8.
recv #2: 1024 bytes
```

Hình A.6. Server xử lý hai Client sessions liên tiếp.

Sau đó Server tiếp nhận phiên thứ hai:

- Client đã kết nối: ('172.20.10.9', 49272)

Hai phiên sử dụng cùng địa chỉ IP Client nhưng có source port khác nhau, cho thấy đây là hai TCP sessions riêng biệt[2].

A.7. Thử nghiệm lỗi khi Client chạy trước Server

Khi Client được chạy trong lúc Server chưa hoạt động, Client không thể thiết lập kết nối TCP và phát sinh:

```
Nhập tin nhắn (gõ 'quit' để thoát): quit
PS C:\NAM3\Lập trình mạng\Nhom09_Baitap2> python echo_client.py
Đang kết nối đến server 127.0.0.1:9009...
Lỗi ConnectionRefusedError: không thể kết nối đến server.
PS C:\NAM3\Lập trình mạng\Nhom09_Baitap2>
```

Hình A.7. Thử nghiệm Client chạy trước Server.

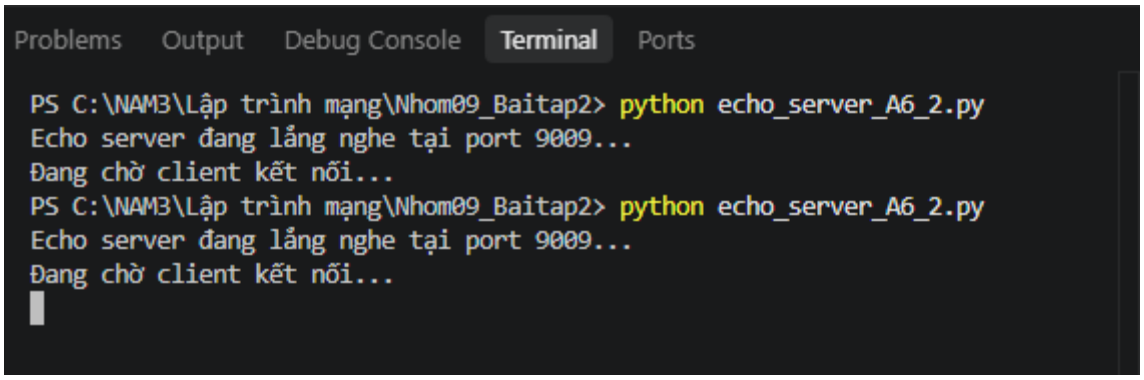
Chương trình bắt riêng ngoại lệ `ConnectionRefusedError` và hiển thị thông báo:

- Lỗi `ConnectionRefusedError`: không thể kết nối đến server.

A.8. Thử nghiệm bỏ SO_REUSEADDR

Trong file `echo_server_A6_2.py`, tùy chọn `SO_REUSEADDR` được sử dụng để thiết lập tùy chọn của socket trước khi `bind()` [1].

Sau khi bảo đảm port 9009 không còn được process khác sử dụng, Server vẫn có thể bind port và khởi động thành công:



Hình A.8. Thử nghiệm Server sau khi loại bỏ `SO_REUSEADDR`.

Server được dừng và khởi động lại ngay sau đó; trong lần thử nghiệm này Server tiếp tục bind thành công và không phát sinh WinError 10048.

Lưu ý: Trong một lần thử nghiệm trước đó, WinError 10048 xuất hiện vì một python.exe khác (PID 2856) vẫn đang giữ port 9009. Sau khi process đó được kết thúc và port được giải phóng, thử nghiệm A6.2 được thực hiện lại thành công. Vì vậy báo cáo không nên dùng lần lỗi đó để kết luận rằng lỗi chắc chắn do việc loại bỏ `SO_REUSEADDR`.

A.9. Thử nghiệm UTF-8 và dữ liệu khoảng 5000 ký tự

A.9.1. Chuỗi tiếng Việt

Client gửi:

- Xin chào Việt Nam! Đây là bài kiểm tra UTF-8.

Kết quả:

- `len(text) = 45`
- `len(text.encode('utf-8')) = 54`

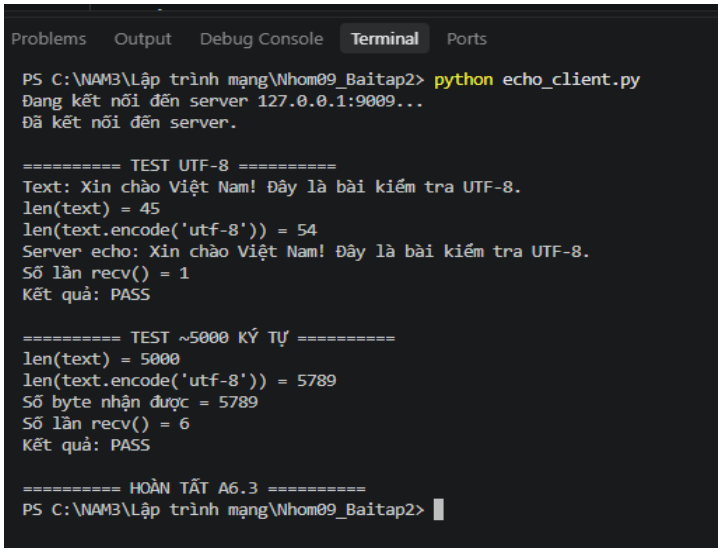
Điều này cho thấy số ký tự và số byte sau khi mã hóa UTF-8 không nhất thiết bằng nhau khi chuỗi chứa ký tự Unicode.

Server Echo lại dữ liệu và Client nhận đúng dữ liệu:

- Kết quả: PASS

A.9.2. Dữ liệu khoảng 5000 ký tự

Kết quả:



Hình A.9. Kết quả kiểm tra UTF-8 và dữ liệu khoảng 5000 ký tự.

Dữ liệu 5000 ký tự được chia thành nhiều lần `recv()` do Buffer Server là 1024 bytes. Điều này minh họa rằng một lần `sendall()` ở phía Client không đảm bảo dữ liệu sẽ được nhận bằng đúng một lần `recv()` [1][2].

A.10. Tổng kết Part A

Qua các thử nghiệm, nhóm đã xây dựng và kiểm tra thành công chương trình Echo Client–Server sử dụng TCP Socket với port 9009.

Các nội dung đã thực hiện gồm:

- Khởi động TCP Echo Server.
- Client kết nối và trao đổi dữ liệu với Server.
- Kiểm tra địa chỉ của listening socket và session socket.
- Kiểm tra trạng thái kết nối bằng netstat.
- Kiểm tra hai TCP sessions liên tiếp.
- Kiểm tra trường hợp Client chạy trước Server.
- Thử nghiệm khi loại bỏ `SO_REUSEADDR`.
- Kiểm tra dữ liệu tiếng Việt bằng UTF-8.
- Kiểm tra dữ liệu khoảng 5000 ký tự và số lần `recv()` = 6.

Phần B - Cài đặt bước đăng nhập của giao thức chat

B.1. Mục tiêu

Phần B được thực hiện dựa trên bản đặc tả giao thức chat mà nhóm đã xây dựng ở Bài tập nhóm số 1. Trong phần này, nhóm chỉ cài đặt bước đăng nhập, chưa thực hiện chức năng nhắn tin.

Nhóm xây dựng hai chương trình `chat_login_server.py` và `chat_login_client.py` bằng Python Socket TCP. Server nhận thông điệp đăng nhập từ client, kiểm tra tên đăng nhập và trả về kết quả tương ứng. Client nhập tên từ bàn phím, gửi yêu cầu LOGIN và hiển thị phản hồi từ server.

Ngoài ra, chương trình sử dụng bộ đệm để xử lý đúng ranh giới thông điệp TCP. Mỗi thông điệp được kết thúc bằng ký tự `\n`.

B.2. Đặc tả thông điệp

Thông điệp được truyền dưới dạng chuỗi UTF-8 và kết thúc bằng `\n`.
Client gửi yêu cầu đăng nhập: `LOGIN|<tên>\n`
Ví dụ: `LOGIN|Nam\n`
Nếu đăng nhập thành công, server trả về:
`OK|Chào Nam\n`
Nếu đăng nhập không thành công, server trả về:
`ERR|<mã>|<mô tả>\n`
Các mã lỗi được sử dụng trong chương trình:

Mã lỗi	Ý nghĩa	Thông báo
ERR|1	Tên đã tồn tại	ERR|1|Tên đã tồn tại
ERR|3	Tên sai định dạng	ERR|3|Tên sai định dạng
ERR|4	Sai trình tự	ERR|4|Sai trình tự: Cần LOGIN trước

B.3. Cài đặt Server

Chương trình `chat_login_server.py` sử dụng TCP Socket và lắng nghe tại cổng 9009 của nhóm[1].

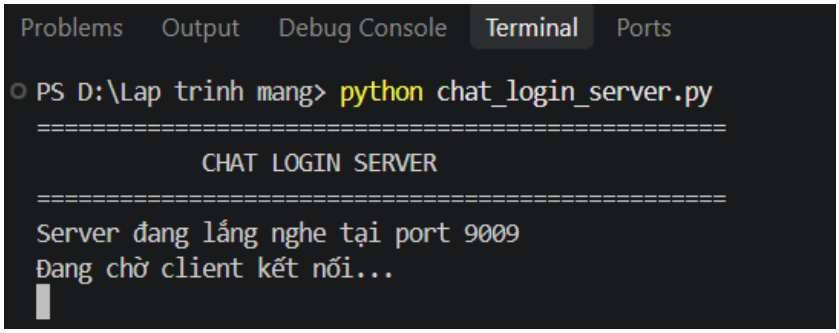
Server duy trì tập hợp `logged_in_names` để lưu các tên đang đăng nhập. Khi Client gửi thông điệp `LOGIN`, server kiểm tra tên theo các điều kiện:

- Tên không được rỗng.
- Tên không dài quá 20 ký tự.
- Không chứa ký tự `|`.
- Không chứa ký tự `\n`.
- Không được trùng với tên đang đăng nhập.

Nếu tên hợp lệ và chưa tồn tại, server thêm tên vào `logged_in_names`, chuyển trạng thái của client sang đã đăng nhập và trả về: `OK|Chào <tên>`

- Nếu tên đã được sử dụng, server trả về: `ERR|1|Tên đã tồn tại`
- Nếu tên không hợp lệ, server trả về: `ERR|3|Tên sai định dạng`
- Server cũng kiểm tra trạng thái đăng nhập. Nếu client chưa LOGIN nhưng gửi lệnh khác, server trả về: `ERR|4|Sai trình tự: Cần LOGIN trước`
- Nếu client đã LOGIN và gửi LOGIN lần nữa, server trả về: `ERR|4|Sai trình tự: Đã đăng nhập`
- Khi nhận QUIT, server gửi: BYE

Sau đó kết thúc phiên. Nếu `recv()` trả về `b""`, server cũng hiểu rằng client đã đóng kết nối. Khi kết thúc phiên, chuyển trạng thái của client sang đã đăng nhập `logged_in_names`.



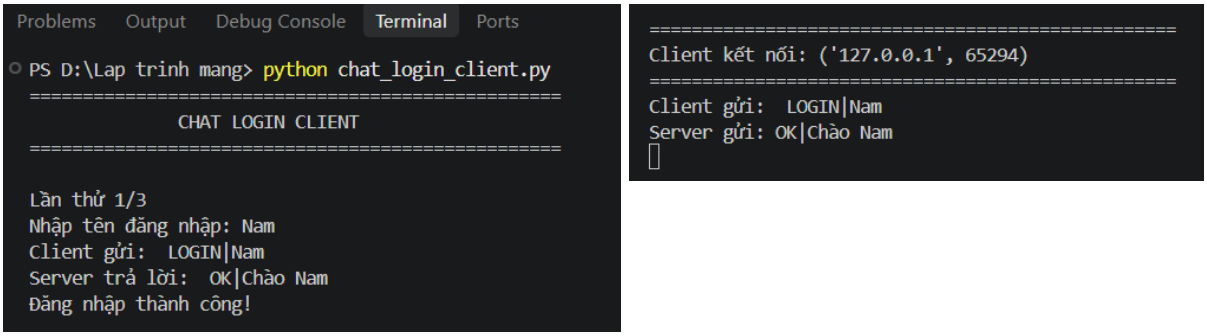
Hình B.3. Server khởi động và lắng nghe kết nối tại cổng 9009.

B.4. Cài đặt Client

Chương trình `chat_login_client.py` kết nối đến server tại 127.0.0.1:9009 [1].

- Client yêu cầu người dùng nhập tên và tạo thông điệp: LOGIN|<tên>
- Sau đó client gửi thông điệp đến server và nhận phản hồi.
- Nếu nhận được OK, client thông báo đăng nhập thành công.
- Nếu nhận được ERR, client cho phép người dùng nhập lại. Số lần thử đăng nhập tối đa là 3 lần. Sau 3 lần không thành công, client kết thúc chương trình.

Sau khi LOGIN thành công, client có thể gửi LOGIN lại để kiểm tra lỗi sai trình tự hoặc gửi QUIT để kết thúc phiên.



Hình B.4. Client gửi yêu cầu LOGIN và nhận phản hồi thành công từ Server.

B.5. Xử lý ranh giới thông điệp

TCP cung cấp một luồng byte (byte stream), do đó một lần gọi `recv()` không đảm bảo nhận được đúng một thông điệp ở tầng ứng dụng [2].Vì vậy, nhóm sử dụng hàm `recv_line()` và biến buffer để tách thông điệp theo ký tự `\n` [1].

Quá trình xử lý gồm:

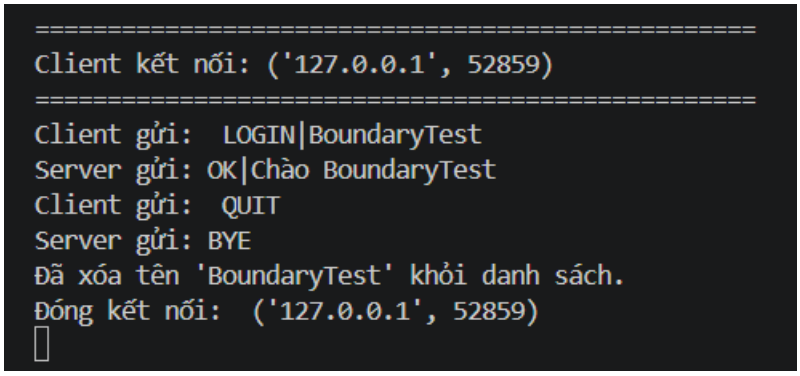
1. Nhận dữ liệu bằng recv(1024).
2. Lưu dữ liệu vào buffer.
3. Kiểm tra xem buffer đã có \n chưa.
4. Dùng partition`(b"\n")` để lấy thông điệp đầu tiên.
5. Giữ lại phần dữ liệu còn lại trong buffer.

Ví dụ:

- client gửi hai thông điệp trong cùng một lần `sendall()`: LOGIN|Nam\nQUIT\n

- Server vẫn tách thành hai thông điệp:
  LOGIN|Nam
  QUIT
- Trong file test_message_boundary.py, nhóm kiểm tra trường hợp này bằng:
  sock.sendall(b"LOGIN|Nam\nQUIT\n")
- Kết quả mong đợi:
  Message 1 từ server: OK|Chào Nam
  Message 2 từ server: BYE

Qua kiểm thử này, nhóm kiểm tra được rằng chương trình không nhầm hai thông điệp được gửi trong cùng một lần sendall().

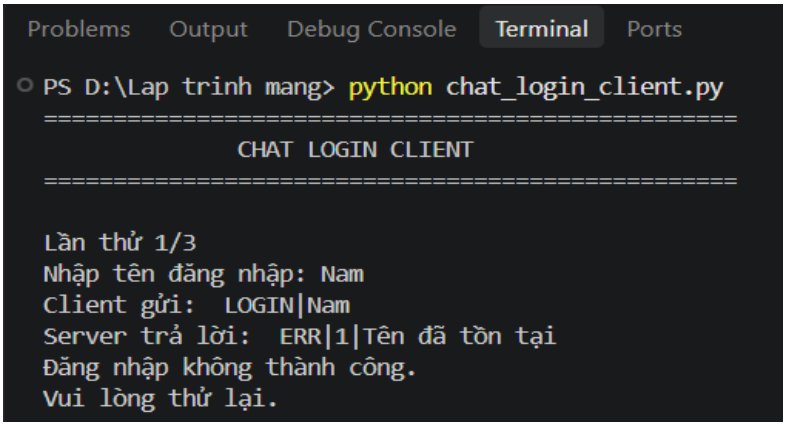


Hình B.5. Kiểm thử ranh giới thông điệp khi Client gửi hai thông điệp trong cùng một lần sendall()

B.6. Kiểm thử và nhận xét.

Bảng 1. Các ca kiểm thử bước đăng nhập

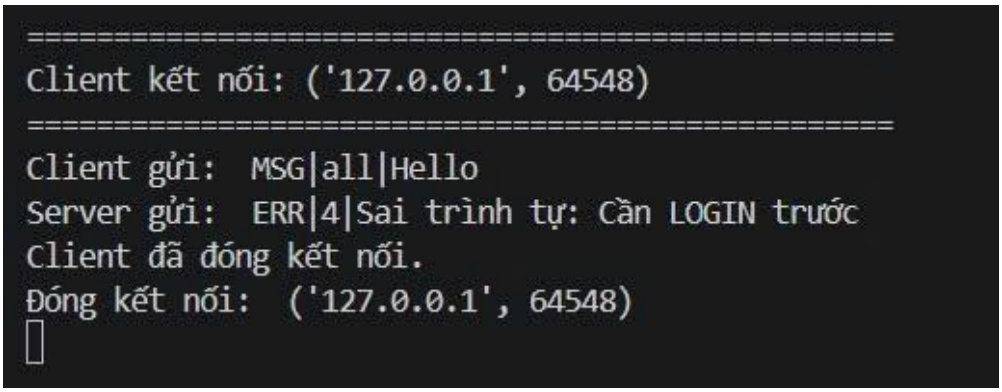
TT	Mục đích	Dữ liệu client gửi	Kết quả mong đợi	Kết quả thực tế / Đạt
1	Đăng nhập hợp lệ	LOGIN|Nam\n	OK|Chào Nam\n	OK|Chào MaiTest\n / Đạt
2	Tên đã tồn tại	LOGIN|Nam\n	ERR|1|Tên đã tồn tại\n	ERR|1|Tên đã tồn tại\n / Đạt
3	Sai trình tự trước LOGIN	MSG|all|Hello\n	ERR|4|Sai trình tự: Cần LOGIN trước\n	ERR|4|Sai trình tự: Cần LOGIN trước\n / Đạt
4	Kiểm tra message boundary	LOGIN|MaiTest\nQUIT\n	Nhận 2 message: OK|Chào MaiTest và BYE	Nhận đúng 2 message / Đạt



Hình B.6.1. Kiểm thử trường hợp tên đăng nhập đã tồn tại.



Hình B.6.2. Kiểm thử tên đăng nhập không hợp lệ.



Hình B.6.3. Kiểm thử trường hợp Client gửi lệnh trước khi LOGIN.

Nhận xét: Qua kiểm thử, chương trình có thể thực hiện bước đăng nhập theo yêu cầu. Server kiểm tra được tên đăng nhập, tên bị trùng và trường hợp Client gửi lệnh sai trình tự. Ngoài ra, cơ chế `buffer` và `recv_line()` giúp Server tách đúng các thông điệp khi nhiều thông điệp được gửi trong cùng một lần `sendall()`.

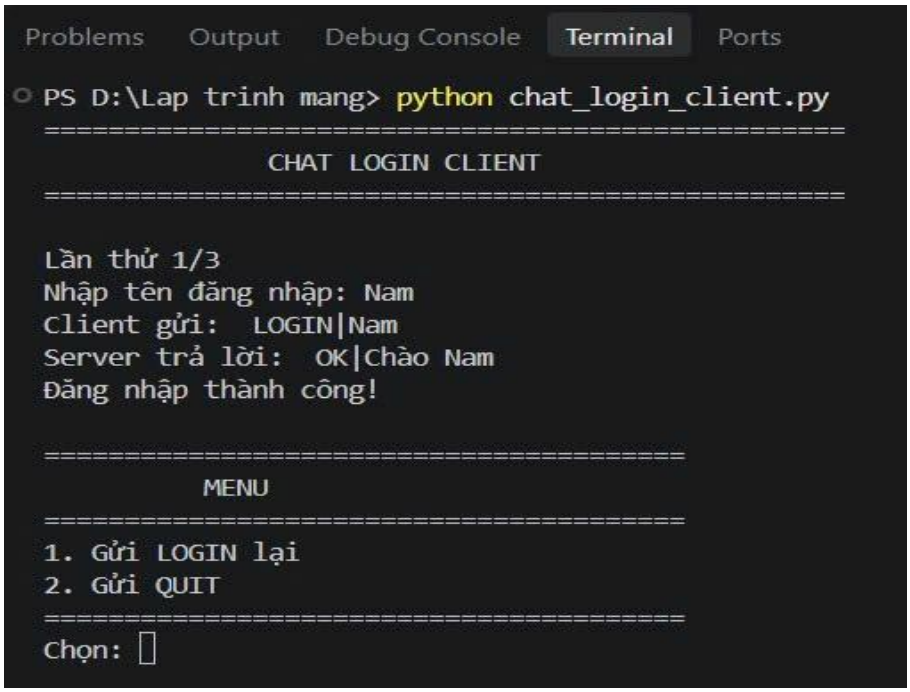
Nhìn chung, phần cài đặt đáp ứng các yêu cầu chính của Phần B. Các kết quả kiểm thử được ghi lại và có thể sử dụng ảnh chụp màn hình để minh họa trong phần phụ lục của báo cáo.

B.7. Kiểm thử hai client cùng kết nối

Nhóm thực hiện kiểm thử với hai Client để quan sát cách Server xử lý nhiều kết nối. Trong phiên bản này, Server xử lý Client theo thứ tự và không sử dụng cơ chế xử lý đồng thời [1].

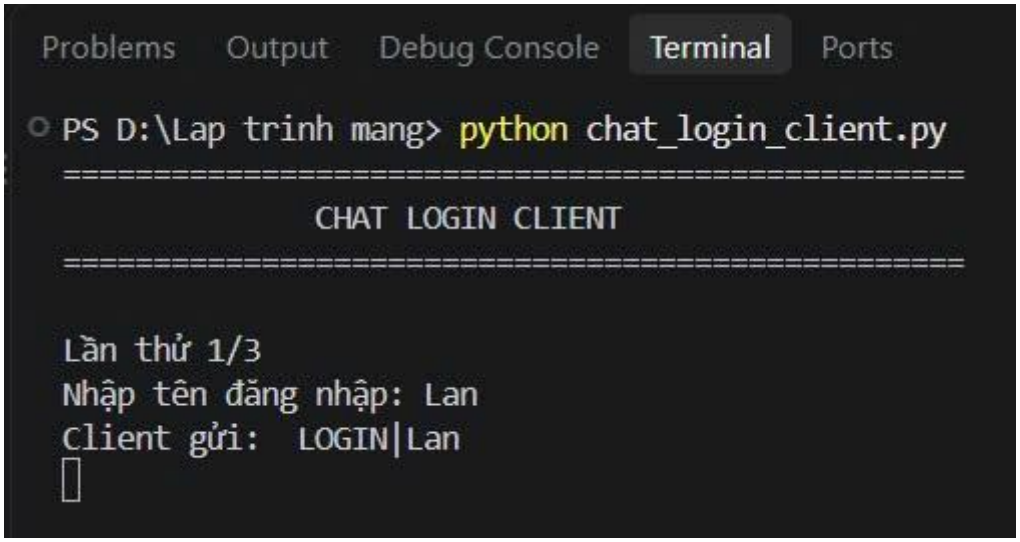
Client thứ nhất được Server chấp nhận và xử lý trước. Trong khi Client thứ nhất vẫn đang duy trì kết nối, Server tiếp tục xử lý các yêu cầu của Client thứ nhất nên chưa chuyển sang xử lý Client tiếp theo.

Sau khi Client thứ nhất gửi `QUIT` và kết thúc phiên, Server quay lại trạng thái chờ kết nối và tiếp tục xử lý Client thứ hai. Qua kiểm thử, nhóm quan sát được Client thứ hai phải chờ cho đến khi phiên làm việc của Client thứ nhất kết thúc.

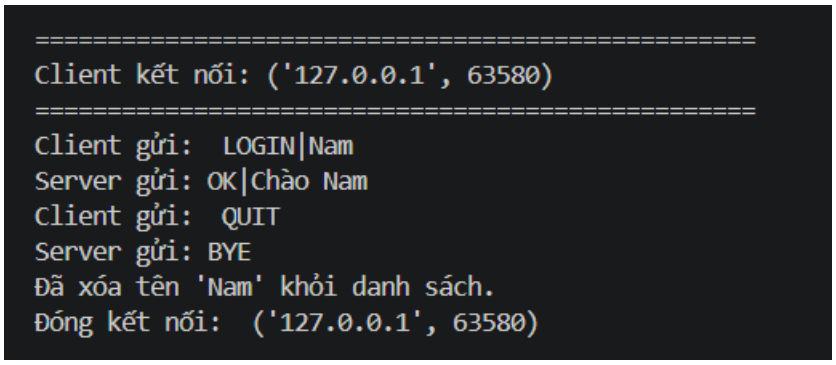


Hình B.7.1. Client thứ nhất đăng nhập thành công và đang duy trì kết nối với Server.

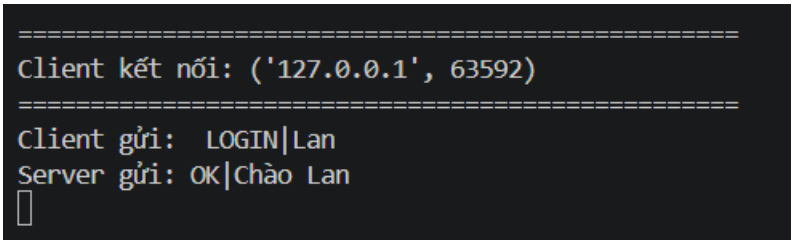




Hình B.7.2. Client thứ hai kết nối nhưng phải chờ trong khi Server đang xử lý Client thứ nhất.



Hình B.7.3. Client thứ nhất gửi QUIT và kết thúc phiên làm việc.



Hình B.7.4. Sau khi Client thứ nhất kết thúc phiên, Server tiếp tục xử lý Client thứ hai.

Phần C – Công cụ tra cứu tên miền dns_lookup.py

C.1 Mục tiêu

Xây dựng chương trình dns_lookup.py bằng Python để tra cứu thông tin DNS của một tên miền. Chương trình nhận tên miền từ tham số dòng lệnh và hiển thị thông tin về máy đang chạy, địa chỉ IPv4, các kết quả IPv4/IPv6 và kết quả tra ngược địa chỉ IP [3][4].

C.2 Nội dung thực hiện

Chương trình thực hiện các chức năng sau:

- Sử dụng `socket.gethostname()` để lấy tên máy đang chạy chương trình [1].
- Sử dụng socket UDP kết nối đến 8.8.8.8 và getsockname() để lấy địa chỉ IP thật của máy trong mạng LAN.
- Sử dụng `socket.gethostbyname()` để lấy địa chỉ IPv4 của tên miền [1].
- Sử dụng `socket.getaddrinfo(domain, 80, type=socket.SOCK_STREAM)` để lấy toàn bộ kết quả địa chỉ IPv4 và IPv6 [1].
- Sử dụng `socket.gethostbyaddr()` để tra ngược địa chỉ IPv4 về tên máy chủ
- Xử lý riêng lỗi socket.gaierror và lỗi OSError [1].

C.3 Kết quả mã nguồn chương trình

```
PS C:\Users\MSII\Downloads\LTM> python dns_lookup.py
Cách dùng: python dns_lookup.py <domain>
```

Hình C.3. Thông báo hướng dẫn sử dụng khi không cung cấp tên miền

C.4 Kết quả chạy chương trình

Chương trình được chạy bằng lệnh:

```
PS C:\Users\MSII\Downloads\LTM> python dns_lookup.py google.com.vn
============================================================
DNS LOOKUP
============================================================
Domain: google.com.vn

[1] Thông tin máy
Hostname: MSI
IPv4: 192.168.1.5

[2] gethostbyname()
IPv4: 172.253.134.94

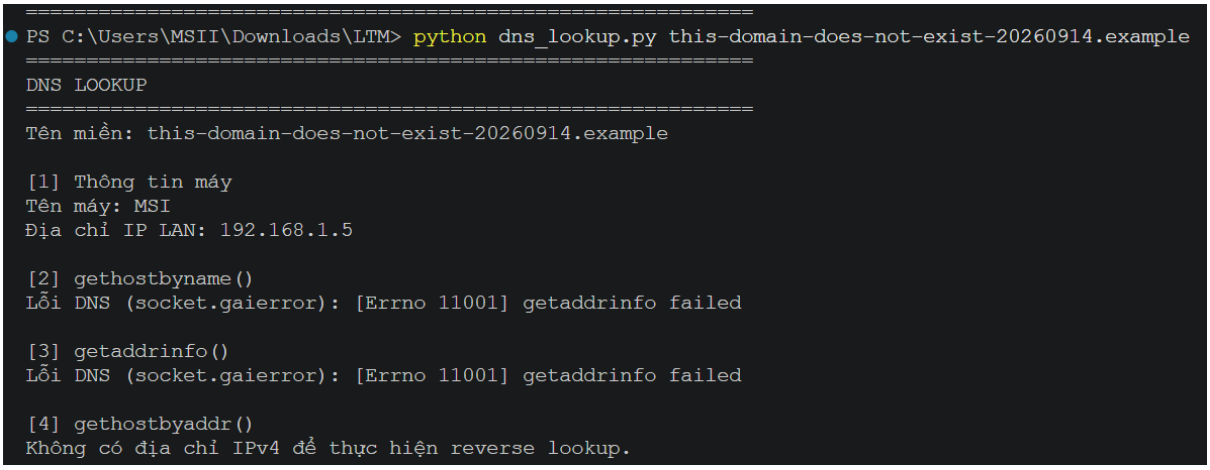
[3] getaddrinfo()
AF_INET: 172.253.134.94

[4] gethostbyaddr()
IP: 172.253.134.94
Hostname: sk-in-f94.1e100.net
Aliases: []
```

Hình C.4 Kết quả chạy chương trình với tên miền google.com.vn

C.5 Kiểm tra tên miền không tồn tại

Chương trình được kiểm tra với một tên miền không tồn tại:



Hình C.5 Kết quả kiểm tra tên miền không tồn tại

Kết quả cho thấy chương trình có thể xử lý lỗi DNS bằng socket.gaierror mà không bị dừng đột ngột [1].

C.6 Bảng 2. Kết quả tra cứu DNS

Tên miền	IPv4	Số dòng getaddrinfo (IPv4 / IPv6)	Tra ngược	Ghi chú
google.com.vn	172.253.134.94	1 (1 / 0)	sk-in-f94.1e100.net	Tên miền .vn
[Tên miền có IPv6]	[IPv4]	[x (x / x)]	[Hostname]	Tên miền quốc tế, có IPv6
this-domain-does-not-exist-20260914.example	Không có 0 (	0 (0 / 0)	Không thực hiện	Tên miền không tồn tại

Nhận xét: Qua quá trình thực hiện, chương trình có thể lấy thông tin của máy và thực hiện các thao tác tra cứu DNS cơ bản. Hàm `gethostbyname()` được sử dụng để lấy địa chỉ IPv4 của tên miền [1]. Hàm `getaddrinfo()` cho phép lấy nhiều kết quả và hỗ trợ cả địa chỉ IPv4 (AF_INET) và IPv6 (AF_INET6). Khi tên miền không tồn tại, chương trình bắt được lỗi `socket.gaierror` và hiển thị thông báo phù hợp. Ngoài ra, `gethostbyaddr()` được sử dụng để tra ngược địa chỉ IP về tên máy chủ [1].

Phần D – Câu hỏi thảo luận

Câu 1. `accept()` sinh ra một socket mới để làm gì? Nếu Server gọi `recv()` ngay trên socket lắng nghe thì điều gì xảy ra và vì sao?

Giải: `accept()` tạo ra một socket mới dành riêng cho Client vừa kết nối để Server trao đổi dữ liệu với Client đó [1]. Socket lắng nghe vẫn tiếp tục có nhiệm vụ chờ các kết nối mới. Trong Phần A, Server lắng nghe tại 0.0.0.0:9009; sau khi Client kết nối, Server sử dụng session socket để giao tiếp với Client. Kết quả `conn.getsockname()` cho biết địa chỉ local của session socket, còn `conn.getpeername()` cho biết địa chỉ của Client. Vì vậy, Server cần sử dụng socket được tạo ra bởi accept() để thực hiện trao đổi dữ liệu với Client.

Câu 2. Bạn gọi `send()` với 100 byte nhưng hàm trả về 20 — điều đó nghĩa là gì? Vì sao trong học phần luôn dùng `sendall()`? Có tình huống nào `sendall()` vẫn chưa đủ để bảo đảm bên nhận đọc đúng thông điệp?

Giải: Nếu `send()` được gọi với 100 byte nhưng chỉ trả về 20 thì lần gọi đó mới xử lý được 20 byte, phần còn lại cần được xử lý tiếp. Vì vậy, chương trình thường sử dụng `sendall()` để cố gắng gửi toàn bộ dữ liệu. Tuy nhiên, `sendall()` không đảm bảo bên nhận sẽ nhận toàn bộ dữ liệu trong một lần `recv()` [1][2]. Trong Phần A, khi kiểm tra dữ liệu khoảng 5000 ký tự, dữ liệu được chia thành nhiều lần `recv()` và kết quả tổng kết ghi nhận `recv() = 6`, do Buffer Server là 1024 bytes. Trong Phần B, nhóm sử dụng buffer và `recv_line()` với ký tự `\n` để xác định ranh giới từng thông điệp. Vì vậy, `sendall()` vẫn cần kết hợp với cơ chế xác định ranh giới thông điệp để bên nhận đọc đúng từng message.

Câu 3. `gethostbyname()` và getaddrinfo() khác nhau ở những điểm nào? Vì sao chương trình dùng thật nên dùng `getaddrinfo()` hoặc `socket.create_connection()`?

Giải: `gethostbyname()` được sử dụng để lấy địa chỉ IPv4 của tên miền. Trong Phần C, chương trình sử dụng hàm này để lấy IPv4 của google.com.vn, với kết quả 172.253.134.94. Trong khi đó, `getaddrinfo()` được sử dụng để lấy các kết quả địa chỉ phục vụ kết nối TCP và hỗ trợ cả IPv4 (AF_INET) và IPv6 (AF_INET6) [1]. Kết quả kiểm tra google.com.vn ghi nhận 1 kết quả getaddrinfo, gồm 1 IPv4 và 0 IPv6. Vì vậy, `getaddrinfo()` linh hoạt hơn khi chương trình cần hỗ trợ nhiều loại địa chỉ. `socket.create_connection()` có thể được dùng để thực hiện kết nối TCP dựa trên thông tin tên miền và cổng.

KẾT LUẬN

Qua bài tập thực hành nhóm số 2, nhóm đã thực hiện và kiểm tra các nội dung cơ bản của lập trình mạng bằng Python, tập trung vào TCP Socket, xây dựng giao thức đăng nhập cho ứng dụng chat và thực hành DNS.

Ở Phần A, nhóm đã xây dựng chương trình Echo Client–Server sử dụng TCP Socket. Nhóm thực hiện kiểm tra thông tin kết nối bằng `getsockname()` và `getpeername()`, quan sát kết nối bằng netstat, đồng thời kiểm tra trường hợp có hai Client kết nối đến cùng một Server. Qua các thử nghiệm, nhóm hiểu rõ hơn về địa chỉ IP, port, socket và cách TCP thiết lập kết nối giữa Client và Server.

Ở Phần B, nhóm tiếp tục sử dụng bản đặc tả giao thức chat đã xây dựng trong Bài tập nhóm số 1 và cài đặt bước đăng nhập. Server có chức năng nhận thông điệp LOGIN, kiểm tra tên đăng nhập, xử lý tên trùng và tên không hợp lệ, đồng thời phản hồi bằng thông điệp OK hoặc ERR. Nhóm cũng sử dụng bộ đệm để xử lý ranh giới thông điệp trong TCP và thực hiện kiểm thử với hai Client. Kết quả kiểm thử cho thấy chương trình xử lý được các trường hợp đăng nhập thành công, lỗi dữ liệu và kết thúc phiên theo yêu cầu.

Ở Phần C, nhóm thực hành các thao tác phân giải tên miền bằng các hàm DNS trong Python như `gethostname()`, `gethostbyname()`, `getaddrinfo()` và `gethostbyaddr()`. Nhóm kiểm tra nhiều trường hợp khác nhau, bao gồm cả trường hợp tên miền không tồn tại, từ đó hiểu được cách chương trình lấy thông tin địa chỉ IP từ tên miền và xử lý lỗi khi phân giải DNS.

Ở Phần D, nhóm tổng hợp và thảo luận một số kiến thức quan trọng liên quan đến `accept()`, `send()`, `sendall()`, ranh giới thông điệp trong TCP và sự khác nhau giữa một số hàm phân giải tên miền. Các nội dung này giúp nhóm liên hệ giữa phần lý thuyết và kết quả thực nghiệm đã thực hiện trong các phần trước.

Trong quá trình làm bài, nhóm có sử dụng AI để hỗ trợ một số vấn đề mà nhóm chưa biết cách xử lý. Dụng AI để tham khảo cách sửa lỗi, giải thích thêm những phần kiến thức chưa hiểu rõ. Tuy nhiên, nhóm vẫn tự chạy thử chương trình, kiểm tra kết quả và điều chỉnh lại nội dung trước khi hoàn thiện báo cáo.

Thông qua bài thực hành, nhóm có thêm kinh nghiệm trong việc lập trình Client–Server, kiểm tra kết nối mạng, thiết kế và cài đặt giao thức ứng dụng đơn giản, xử lý lỗi và kiểm thử chương trình. Đây cũng là cơ sở để nhóm tiếp tục phát triển các chức năng khác của ứng dụng chat trong các bài thực hành tiếp theo.

TÀI LIỆU THAM KHẢO

[1] Python Software Foundation, “socket — Low-level networking interface,” *Python 3 Documentation*. Available at: https://docs.python.org/3/library/socket.html

[2] W. Eddy, Ed., “Transmission Control Protocol (TCP),” RFC 9293, Internet Engineering Task Force (IETF), 2022. Available at: https://www.rfc-editor.org/rfc/rfc9293.html

[3] P. Mockapetris, “Domain Names — Concepts and Facilities,” RFC 1034, Internet Engineering Task Force (IETF), 1987. Available at: https://www.rfc-editor.org/rfc/rfc1034

[4] P. Mockapetris, “Domain Names — Implementation and Specification,” RFC 1035, Internet Engineering Task Force (IETF), 1987. Available at: https://www.rfc-editor.org/rfc/rfc1035

[5] J. Kurose and K. Ross, *Computer Networking: A Top-Down Approach*, 8th ed. Pearson, 2021.

[6] B. Rhodes and J. Goerzen, *Foundations of Python Network Programming*, 3rd ed. Apress, 2014.

[7] Slide bài giảng Chương 3 – Lập trình ứng dụng hướng kết nối TCP

PHỤ LỤC

Bảng phân công công việc

TT	Họ và tên	Mã sinh viên	Công việc đảm nhận	Đóng góp (%)
1	Phimmasone Khamphouvanh	3120224192	Nhóm trưởng: Thực hiện Phần A: Echo Client–Server, kiểm tra kết nối TCP Socket, getsockname(), getpeername() và netstat.	23
2	Bounsavath Ameena	3120224190	Thực hiện Phần C: Phân giải DNS, thực hành các hàm gethostname(), gethostbyname(), getaddrinfo() và gethostbyaddr().	22
3	Lattanaphimphone Chindaphet	3120224191	Thực hiện Phần B: Cài đặt bước đăng nhập của giao thức chat, xử lý mã lỗi, buffer và kiểm thử Client–Server. Kiểm tra, chỉnh sửa và hoàn thiện báo cáo	23
4	Silaphet Thit	3120224188	Thực hiện Phần D: Câu hỏi thảo luận, phân tích accept(), send(), sendall() và các hàm phân giải tên miền.	22
5	Thepsoulivong Syvone	3120224193	Thực hiện Phần D: Câu hỏi thảo luận.	10

Bảng Nhật ký lỗi

TT	Thông báo lỗi (chép nguyên văn)	Tình huống xuất hiện	Nguyên nhân	Cách khắc phục
1	ConnectionRefusedError	Client chạy trước khi Server được khởi động	Server chưa hoạt động và chưa lắng nghe tại port 9009	Khởi động Server trước, sau đó chạy Client
2	WinError 10048	Khi chạy Server nhưng port 9009 đang được process khác sử dụng	Một tiến trình python.exe khác đang giữ port 9009	Kiểm tra và kết thúc process đang sử dụng port 9009, sau đó chạy lại Server
3	socket.gaierror	Khi kiểm tra với tên miền không tồn tại	Không thể phân giải tên miền	Bắt ngoại lệ socket.gaierror và hiển thị thông báo phù hợp